

Object Oriented Programming Project

STUDY ANXIETY CALCULATOR

Subject: Object Oriented Programming (C++)

Project Type: Console-Based Application

Concept: Study Anxiety Assessment

Submitted By:

Anamta Khurram (SP25-BSSE-0033)

Submitted To:

Miss Aiman Fatima

Department:

Software Engineering

1. INTRODUCTION

The *Study Anxiety Calculator* is a console-based application developed in **C++** using **Object Oriented Programming (OOP)** principles. The goal of this project is to assess a student's anxiety related to studies, exams, and academic pressure through a quiz-based system.

The program collects user responses, calculates an anxiety score, categorizes the anxiety level, provides personalized study suggestions, and gathers user feedback. This project demonstrates real-world implementation of OOP concepts including encapsulation, inheritance, abstraction, polymorphism, and composition.

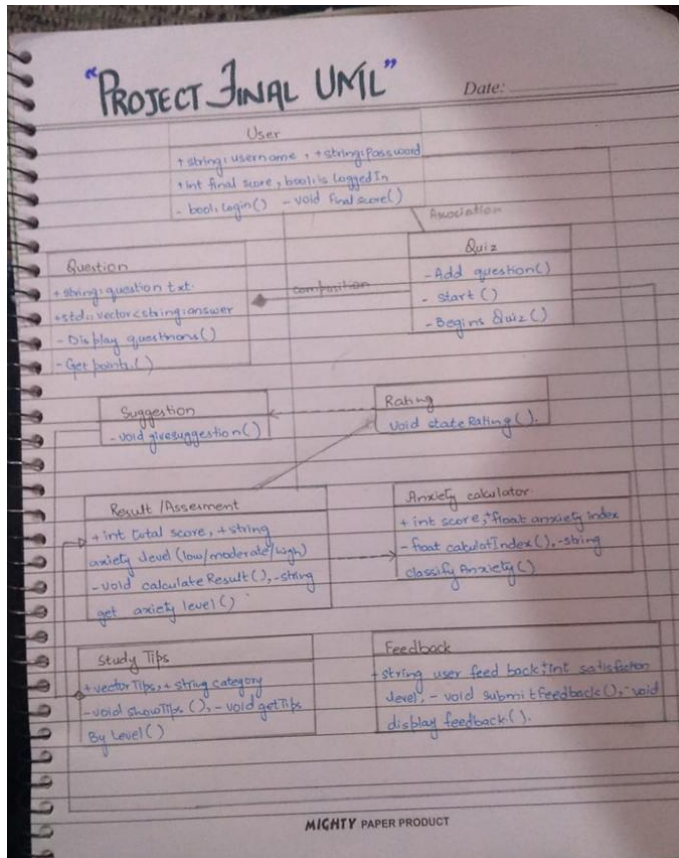
2. CORE CONCEPT OF THE PROJECT

The system works as an interactive console program where:

- Users create an account and log in
- A quiz related to study anxiety is conducted
- Each answer carries a predefined score
- Total anxiety score is calculated
- Anxiety level is classified as **Low, Moderate, or High**
- Study suggestions are displayed

- User feedback is collected

3. UML DIAGRAM



Classes Included:

- User
- Question
- Quiz
- Assessment (Abstract)
- Result
- Suggestion
- Feedback

The UML diagram shows **inheritance**, **association**, and **composition** relationships among classes, ensuring modular and reusable code.

4. CLASSES:

User Class

Handles user registration and authentication.

Attributes:

- Username
- Password

Functions:

- signUp()
- signIn()

This class ensures secure access to the system.

Question Class

Represents a single quiz question.

Attributes:

- Question text
- Options (vector)
- Scores (vector)

Functions:

- display()
- getScore()

Encapsulation ensures data safety.

Quiz Class

Manages the entire quiz process.

Attributes:

- Vector of Question objects
- Total score

Functions:

- addQuestion()
- start()
- getTotalScore()
- getMaxScore()

Demonstrates **composition**.

Assessment Class (Abstract)

Defines a base structure for result evaluation.

Function:

- calculateResult() (pure virtual)

Implements **abstraction**.

Result Class

Inherits from Assessment and calculates anxiety percentage.

Functions:

- calculateResult()

Demonstrates **inheritance and polymorphism**.

Suggestion Class

Provides study and stress-management advice based on anxiety level.

Feedback Class

Collects user satisfaction rating and comments.

Includes:

- Input validation
- Exception handling

5. OOP CONCEPTS USED:

Encapsulation

Data members are kept private and accessed through public methods.

Abstraction

Implemented using an abstract class (Assessment).

Inheritance

Result class inherits from Assessment.

Polymorphism

Overridden calculateResult() function.

Composition

Quiz class contains multiple Question objects.

6. USE OF VECTORS:

Vectors are used to:

- Store questions
- Store answer options
- Store scores dynamically

Advantages:

- Automatic resizing
- Efficient memory management
- Easy access and modification

7. EXCEPTION HANDLING:

Exception handling ensures:

- Valid input for feedback ratings (1-5)
- Program stability
- Error-free execution

Implemented using try-catch blocks.

8. CONCLUSION

The *Study Anxiety Calculator* is a well-structured OOP-based C++ project that applies OOP principles. The system is modular, user-friendly, and demonstrates real-world problem-solving using programming concepts. The project provides a practical solution for assessing study-related anxiety by using a structured quiz system, calculating anxiety levels, and offering personalized study and stress-management suggestions.

CODE:

```
#include <iostream>

#include <vector>

#include <string>

#include <stdexcept>

using namespace std;

//creating first main class

class User {

private:

    string username;

    string password;

    //function to sign up

public:

    void signUp() {

        cout << "SIGN UP TO ENTER THE PROGRAM";

        cout << "\nCreate Username:";

        cin >> username;

        cout << "Create Password:";

        cin >> password;

        cout << "Your account has been created successfully!";

    }
```

```

//sign in function
bool signIn() {
    string u, p;
    //entering your personal details according to your signup details
    cout << "\nSIGN IN:";
    cout << "Username:";
    cin >> u;
    cout << "Password:";
    cin >> p;
    if (u == username && p == password) {
        cout << "\nLogin Successful!";
        return true;
    }
    else {
        cout << "Invalid Username or Password!";
        return false;
    }
}

};

//Second main class blue print of the program
class Question {
private:    //private members of variables cant be accessible without any encapsulation
outside of class
    string text;
    vector<string> options;
    vector<int> scores;
public:
    Question(string t, vector<string> o, vector<int> s) { //creating constructor to get an access
to the private values of the class

```

```

    text = t;

    options = o;

    scores = s;
}

void display() {
    cout << "\nEnter your question" << text << endl;

    for (int i = 0; i < options.size(); i++) {
        cout << i + 1 << ". " << options[i] << endl;
    }
}

int getScore(int choice) {
    return scores[choice - 1];
}

};

//Third class to manage the quiz
class Quiz {
private:
    vector<Question>questions;

    int totalScore;

    //we used vector values because vector is used to store multiple values for one answers
public:
    Quiz() { //constructor
        totalScore = 0;
    }

    //creating a function to add questions in the quiz
    void addQuestion(Question q) {
        questions.push_back(q);
    }
}

```



```

//creating a function to start the quiz
void start() {
    int choice;

    //for loop for questions
    for (int i = 0; i < questions.size(); i++) {
        questions[i].display();

        //do-while loop for choices
        do {
            cout << " Choose (1-4): ";

            cin >> choice;

        } while (choice < 1 || choice > 4);

        totalScore += questions[i].getScore(choice);
    }
}

//function to get totalscore
int getTotalScore() {
    return totalScore;
}

//function to get maximum score
int getMaxScore() {
    return questions.size() * 4;
}

};

//creating assesment class
class Assessment {
protected:    //protecting values of this class
    int score;

    int maxScore;

```

public:

```
Assessment(int s, int m) { //constructor
```

```
    score = s;
```

```
    maxScore = m;
```

```
}
```

```
virtual void calculateResult() = 0;
```

```
};
```

//Result class

```
class Result :public Assessment { //Inheritance
```

public:

```
Result(int s, int m) :Assessment(s, m) {}
```

```
//creating function to calculate result
```

```
void calculateResult() {
```

```
    float percentage = (float)score / maxScore * 100;
```

```
    //output for the result
```

```
    cout << "\nRESULT:";
```

```
    cout << "\nScore:" << score << "/" << maxScore;
```

```
    cout << "\nAnxiety Percentage:" << percentage << "%";
```

```
    //condition for anxiety level
```

```
    if (percentage <= 40)
```

```
        cout << "\nAnxiety Level: LOW";
```

```
    else if (percentage <= 70)
```

```
        cout << "\nAnxiety Level: MODERATE";
```

```
    else
```

```
        cout << "\nAnxiety Level: HIGH\n";
```

```
}
```

```

};

//Suggestion class according to anxiety level
class Suggestion {
public:
    void showSuggestion(int score, int maxScore) {
        float percent = (float)score / maxScore * 100;
        //Study Suggestion According to calculator
        cout << "\nSTUDY SUGGESTIONS:";
        //Suggestion According to your level
        if (percent >= 70) {
            cout << "\nReduce study time this is creating more panic" << endl;
            cout << "Practice mind relaxation techniques" << endl;
            cout << "Eat Dark chocolate" << endl;
        }
        else if (percent >= 40) {
            cout << "Improve time management" << endl;
            cout << "Take short breaks" << endl;
            cout << "Avoid last-minute study" << endl;
        }
        else {
            cout << "Keep following your routine" << endl;
            cout << "\nStay consistent" << endl;
        }
    }
};

//class for feedback collecting from user
class Feedback {
private:

```

```
int satisfactionLevel;

string comment;

//function to collect feedback
public:

void collectFeedback() {

    cout << "\nFEEDBACK:";


    while (true) {

        try {

            cout << "\nRate your satisfaction (1-5): ";

            cin >> satisfactionLevel;

            if (cin.fail()) {

                throw runtime_error("Invalid input! Please enter a number between 1 and 5.");

            }


            if (satisfactionLevel < 1 || satisfactionLevel > 5) {

                throw out_of_range("Satisfaction level must be between 1 and 5.");

            }


            break; // valid input, exit loop

        }

        catch (exception& e) {

            cout << e.what() << endl;

            cin.clear();          // clear error flag

            cin.ignore(1000, '\n'); // discard invalid input

        }

    }

}
```

```

    cout << "Any comments (words only): ";
    cin.ignore(); // clear leftover newline
    getline(cin, comment);

    // Ensure comment is not just digits
    bool onlyDigits = true;
    for (char c : comment) {
        if (!isdigit(c) && !isspace(c)) {
            onlyDigits = false;
            break;
        }
    }

    if (onlyDigits) {
        cout << "Comment must contain words, not just numbers. Setting comment as 'No
valid comment'.";
        comment = "No valid comment";
    }

    cout << "Thank you for your feedback!" << endl;
}

void displayFeedback() {
    cout << "\nUSER FEEDBACK: ";
    cout << "\nSatisfaction Level:" << satisfactionLevel << "/5";
    cout << "\nComment: " << comment;
}
};

```

```

//MAIN FUNCTION

int main() {

    cout << "STUDY ANXIETY CALCULATOR" << endl;

    User student;

    student.signUp();

    //condition

    if (!student.signIn())

        return 0;

    //object for quiz

    Quiz quiz;

    vector<string> options;

    vector<int> scores;

    //choice

    options.push_back("Never");
    options.push_back("Rarely");
    options.push_back("Often");
    options.push_back("Always");

    //scores

    scores.push_back(1);
    scores.push_back(2);
    scores.push_back(3);
    scores.push_back(4);

    //Added Questions

    quiz.addQuestion(Question("\nDo you feel stressed during studies?", options, scores));
    quiz.addQuestion(Question("\nDo exams disturb your sleep?", options, scores));
    quiz.addQuestion(Question("\nDo deadlines make you anxious?", options, scores));
    quiz.addQuestion(Question("\nDo you feel tired while studying?", options, scores));
    quiz.addQuestion(Question("\nDo you fear failure in exams?", options, scores));

```

```

    quiz.addQuestion(Question("\nDo you feel pressure to complete syllabus on time?",
options, scores));

    quiz.addQuestion(Question("\nDo you compare your performance with others?", options,
scores));

    quiz.addQuestion(Question("\nDo assignments increase your stress level?", options,
scores));

    quiz.start();

    //getting result for the quiz
    Result result(quiz.getTotalScore(), quiz.getMaxScore());

    result.calculateResult();

    //give suggestions
    Suggestion s;

    s.showSuggestion(quiz.getTotalScore(), quiz.getMaxScore());

    //give feedback
    Feedback f;

    f.collectFeedback();

    f.displayFeedback();

    //end

    cout << "\nThank you for using Study Anxiety Calculator!" << endl;

    return 0;

}

```